Lean-verified instances of the lonely runner conjecture for integer speeds in $[1, 2m]$

Vico Bonfioli

18 October 2026. Preprint, not peer reviewed.

Abstract

For a positive integer m let $V \subset [1, 2m]$ be a set of m distinct positive integers. We consider the inequality $\mathrm{ML}(V) \geq 1/(m+1)$, the lonely runner conjecture for m speeds restricted to speeds of size at most $2m$. We give a formalization in Lean 4 of this inequality for every $m \geq 2$ such that $2m+1$ is prime, for each of the 33 values $m \leq 62$ with $2m+1$ composite, and for 34 further values $64 \leq m \leq 126$ with $2m+1$ composite (listed in Tables 1 and 2). Consequently the statement is verified for every m with $2 \leq m \leq 126$ except $m \in \{94, 97, 103, 109, 115, 122\}$, for which no Lean module has been built; nothing is claimed for $m \geq 127$ except when $2m+1$ is prime. The prime case is a short consequence of a divisibility lemma of Tao and a coprime-pair witness of Bohman and Peng, used at modulus $2m+1$. The composite cases are certified by finite objects of two kinds: weights on witness times (a fractional blocking certificate) and covers of $[1, 2m]$ by parts with capacities (a cover-capacity certificate). The certificates for $m \leq 62$ were found by integer programming; those for $m \geq 64$ by a local search in Rust and were accepted by a second, independent Rust checker before the Lean build. All are checked by the Lean kernel. The window has height $2m$, whereas the known reductions of the conjecture require exponential height, so nothing here bears on the conjecture for all m.

1 Setting, labels and results

Write $\|x\| = \mathrm{dist}(x, \mathbb{Z})$. For $v = (v_1, \ldots, v_m) \in \mathbb{Z}^m_{>0}$ put $\mathrm{gap}_v(t) = \min_i \|v_i t\|$ and $\mathrm{ML}(v) = \sup_{t \in \mathbb{R}} \mathrm{gap}_v(t)$. The lonely runner conjecture (LRC) for m speeds is $\mathrm{ML}(v) \geq 1/(m+1)$ for all v; there are $m+1$ runners, one of them stationary. The *floor* is the value $1/(m+1)$. The *dense window* statement for m is: $\mathrm{ML}(v) \geq 1/(m+1)$ whenever the v_i are distinct and lie in $[1, 2m]$. Tao names the window $[1, 2m]$ as a natural target [10]; it is Problem 6 of the survey [5]. In Lean, `gap` and `ML` are as above for `Fin (k+1)` $\to \mathbb{Z}$ with $m = k+1$ (module `MaxLoneliness`).

Labels. VERIFIED: formalized in Lean 4 with Mathlib, build clean, no `sorry`, axioms among `propext`, `Classical.choice`, `Quot.sound`. PROVED: a complete written proof, not formalized. HEURISTIC: computation or plausibility only. CONJECTURE: no proof. A result has the weakest label in its dependency chain. Where the project record contains no axiom print for a declaration, this is stated.

Results.

- Theorem 3: dense window for $2m+1$ prime. VERIFIED.
- Proposition 4: fractional blocking criterion; instances $m = 4, 7, 12, 13$. VERIFIED.

- Theorem 5: cover-capacity criterion. VERIFIED.
- Theorem 10: dense window for the composite values of $2m+1$ listed in Table 2. VERIFIED per row. For the rows marked S in the table the raw build and axiom output is archived; for the rows marked L the build and the axiom print are recorded in `LEAN_MAP.md` and in development notes only, and were not re-observed when the archive was assembled.
- Statements 6–9: structure used by the certificate search. Labels HEURISTIC, HEURISTIC, PROVED, PROVED respectively. None of them is used by Theorems 3–10, whose proofs depend only on the Lean-checked certificates.

Section 7 lists what is not claimed. In particular Theorem 3 is a routine consequence of known ingredients (Section 8).

2 The prime case

Let $N = 2m + 1$, $m \geq 2$, with N prime, and let V be a multiset of m integers in $[1, 2m]$. No distinctness is needed.

Lemma 1 (Witnesses, VERIFIED)**.** *Let p, q be positive integers. If $q \leq (m+1)\min(v_i p \bmod q,\ q - (v_i p \bmod q))$ for all i, then $\mathrm{gap}_v(p/q) \geq 1/(m+1)$. In particular, if some $q \in [2, m+1]$ divides no v_i, then $\mathrm{ML}(v) \geq 1/(m+1)$ (take $p = 1$).*

Lean: `cube_floor_of_witness`, `floor_of_not_dvd_le`. The second statement is the case used by Tao's Lemma 5.1(i) [10]: if $\mathrm{ML}(v) < 1/(m+1)$ and $v_i \leq 2m$, every $j \in [2, m+1]$ divides some v_i.

Lemma 2 (Pair sum at a prime modulus, VERIFIED)**.** *Let $N = 2m + 1$ be prime, $v_i \in [1, 2m]$, and suppose $v_a + v_b = N$ for two indices $a \neq b$. Then $\mathrm{ML}(v) \geq 1/(m+1)$.*

Proof. No v_i is divisible by N. For $c \in (\mathbb{Z}/N)^\times$ the time c/N satisfies $\|v_i c/N\| \geq 2/N > 1/(m+1)$ unless $v_i c \equiv \pm 1 \pmod N$, that is unless $c \equiv \pm v_i^{-1}$. Each v_i forbids at most two of the $2m$ nonzero residues c. Since $v_b \equiv -v_a$, the indices a, b forbid the same two residues, so at most $2(m-1)$ residues are forbidden and some c survives. □

Lean: `floor_of_pairsum_prime` (module `Delta2Pigeonhole`), whose docstring describes this multiplier count. The text above is a rendering of that argument. For N prime the argument is the witness of Bohman and Peng at sum $s + t = N$ (Section 8).

Theorem 3 (VERIFIED)**.** *Let $m \geq 2$ with $2m + 1$ prime, and $v_1, \ldots, v_m$ integers in $[1, 2m]$. Then $\mathrm{ML}(v) \geq 1/(m+1)$.*

Lean: `dense_floor_of_prime` (module `Delta2DenseWindow`). The explicit axiom list for this declaration is not among the archived prints; it is obtained by `#print axioms LRSpectrumFull.dense_floor_of_prime`.

Proof. If no v_i equals 1, then $v_i \in [2, 2m]$ gives $\|v_i/(2m+2)\| = \min(v_i, 2m+2-v_i)/(2m+2) \geq 2/(2m+2) = 1/(m+1)$, so $t = 1/(2m+2)$ works. If some $q \in [2, m+1]$ divides no v_i, Lemma 1 applies. Otherwise 1 occurs and every $q \in [2, m+1]$ divides some v_i. The only multiple of $m+1$ in $[1, 2m]$ is $m+1$, so some $v_b = m+1$. The multiples of m in $[1, 2m]$ are m and $2m$. If some $v_a = m$, then $v_a + v_b = 2m+1$. Otherwise some $v_a = 2m$ and $1 + 2m = 2m+1$ with the index of the value 1. In both cases the indices differ and Lemma 2 applies. □

3 Certificates for composite $2m+1$

A velocity set $V \subset [1, 2m]$ *has no witness* if for every time $t = p/q$ some $x \in V$ lies in the bad set $B(p/q) = \{x \in [1, 2m] : \|xp/q\| < 1/(m+1)\}$. Thus V is a transversal of the hypergraph of bad sets. By Lemma 1, a witness gives the floor, so it suffices to show that no m-set is a transversal. In Lean, `goodAt k x p q` is the Boolean $q \le (k+2)\min(xp \bmod q,\ q - xp \bmod q)$, and certificate times are given as pairs (p_j, q_j) with a finite set `bad j` that contains every $x \in [1, 2m]$ failing `goodAt` at (p_j, q_j).

Fractional blocking. For a transversal V of size m and weights $y_j \ge 0$ on the times with $\sum_j y_j\,[x \in B_j] \le D$ for every $x \in [1, 2m]$,

$$\sum_j y_j \le \sum_j y_j\,|B_j \cap V| = \sum_{x \in V} \mathrm{load}(x) \le mD.$$

Hence $\sum_j y_j > mD$ excludes transversals of size m. This is weak LP duality for the covering number of the hypergraph, scaled to natural numbers. The principle that an arbitrary weighting of times, that is a finitely supported measure, may be tested against a family is Bedert's Lemma 2.1 [2], and his Lemma 5.3 and Corollary 5.4 use the uniform measure on the times j/p for a prime p dividing no speed; the weights above are an instance of the former.

Proposition 4 (VERIFIED)**.** *Let $k+1 = m$, let $y : \{1..N\} \to \mathbb{N}$, $D \in \mathbb{N}$, and assume* $\mathrm{load}(x) \le D$ *for every $x \in [1, 2m]$ and $mD < \sum_j y_j$. Then every v with $0 < v_i \le 2m$ satisfies* $\mathrm{ML}(v) \ge 1/(m+1)$*. In particular, no distinctness is needed.*

Lean: `floor_of_frac_blocking` (module `Delta2FracBlocking`). Instances `dense_floor_m4`, `m7`, `m12`, `m13` (Table 1).

m	$2m+1$	times	denominator D	total weight
4	3^2	5	1	$5 > 4$
7	$3 \cdot 5$	9	2	$15/2 > 7$
12	5^2	15	2	$25/2 > 12$
13	3^3	21	11	$145/11 > 13$

Table 1: Fractional blocking certificates. The columns are regenerated by the exact-integer program `code/dense/src/fb`, which reads the weights out of the Lean source of `Delta2FracBlocking`, checks that every load is at most D and that the total weight exceeds mD, and rejects four perturbed certificates (`data/dense/fb_check.out`, `fb_selftest.out`). For $2m+1 = 21, 33, 49$ the fractional bound equals m and cannot close the case (docstring of `Delta2CoverCapacity`); this statement is quoted and was not re-derived.

Cover capacity. Let $W = [1, 2m] \setminus V$, so $|W| = m$. If V has no witness then W is *independent*: it contains no bad set B_j (otherwise the time p_j/q_j would be a witness for V, since $V \cap B_j = \emptyset$). Let $U_1, \dots, U_t$ cover $[1, 2m]$ and let c_i be such that every $T \subseteq U_i$ with $|T| > c_i$ contains some B_j. Then

$$m = |W| \le \sum_i |W \cap U_i| \le \sum_i c_i.$$

So $\sum_i c_i < m$ is a contradiction. The c_i are Chvátal–Gomory type cuts: a part of six velocities with capacity three is invisible to the LP, which weighs each vertex at $1/2$.

Theorem 5 (VERIFIED)**.** *Let $v : \mathrm{Fin}(k+1) \to \mathbb{Z}$, $m = k+1$, be positive and injective with values in $[1, 2m]$. Suppose finitely many times (p_j, q_j) with sets $\mathtt{bad}_j$ as above, parts $U_i \subseteq \mathbb{N}$ with $[1, 2m] \subseteq \bigcup_i U_i$, capacities c_i such that every $T \subseteq U_i$ with $|T| > c_i$ contains some $\mathtt{bad}_j$, and $\sum_i c_i < m$. Then $\mathrm{ML}(v) \geq 1/(m+1)$.*

Lean: `floor_of_cover_capacity` (module `Delta2CoverCapacity`). Parts may overlap. Injectivity is a hypothesis here (it is used to count $|W| = m$).

Zero slack. Let $\mathrm{cap}(S)$ be the largest subset of S containing no minimal bad set. By Statement 6 below (HEURISTIC, per m), every cover has $\sum \mathrm{cap}(U_i) \geq m-1$ for the relevant m. A certificate therefore has $\sum c_i = m-1$ exactly, which is what every row of Table 2 has.

4 Structure used by the search

Notation: $Y = \{m+2, \ldots, 2m\}$ ($m-1$ elements), $\mathrm{Low} = \{1, \ldots, m+1\}$. Minimal bad sets are the inclusion-minimal sets among the $B(p/q)$. The velocities 1 and $m+1$ carry singleton minimal bad sets, so they are covered by a part $\{1, m+1\}$ of capacity 0 and removed; write $\mathrm{Low}' = \{2, \ldots, m\}$. These statements guide the search and give no input to Theorems 3–10.

Proposition 6 (HEURISTIC)**.** *For each m tested, Y contains no minimal bad set. Hence $y = \mathbf{1}_Y$ is a feasible dual, every cover satisfies $\sum_i \mathrm{cap}(U_i) \geq m-1$, and in a cover of cost exactly $m-1$ each Y-element is covered once and $\mathrm{cap}(U_i) = |U_i \cap Y|$.*

The first sentence is an exact integer check on the enumerated list of minimal bad sets. It was repeated for $2 \leq m \leq 62$ by the program `code/dense/src/yind`, which enumerates every bad set exactly (breakpoints and midpoints of the arrangement over all real times) and tests whether any bad set lies inside Y, and for $4 \leq m \leq 62$ by the test `y_independent` of `code/dense/src/cg` (command `cg edges M`); the two programs share no code and agree on the number of distinct bad sets, the number of minimal bad sets and the verdict at all 59 values $4 \leq m \leq 62$ (`data/dense/yind_2_62.out`, `cg_edges_4_62.out`; as a negative control `yind control` finds a bad set inside $\{m+1, \ldots, 2m\}$). The search programs `g1` assert the property at run time for each $m \geq 64$. I have no proof for general m. The consequences follow from it by a short argument that has not been adversarially reviewed and is not reproduced here.

Proposition 7 (HEURISTIC)**.** *Given Statement 6, a cover of cost $m-1$ is a packing of* blocks *(T_j, L_j) with $T_j \subseteq Y$ pairwise disjoint, $L_j \subseteq \mathrm{Low}'$ with $\bigcup L_j = \mathrm{Low}'$, and $\tau(T_j \cup L_j) = |L_j|$, where τ is the minimum size of a hitting set of the minimal bad sets contained in $T_j \cup L_j$. Leftover Y-elements are singleton parts of capacity 1.*

Validity of a block is downward closed in L and monotone increasing in T (no separate proof is given here). The equivalence "$\alpha(S) = |T|$ iff $\tau(S) = |L|$" is quoted from development notes that are not archived, without a proof.

Proposition 8 (PROVED, short, plus COMPUTED self-test)**.** *Call a valid block (T, L)* kept *if L is maximal for T and T is minimal for L. Every valid block (T_0, L_0) is dominated ($T \subseteq T_0$, $L \supseteq L_0$) by a kept block. Consequently the family of kept blocks with $|T| \leq 4$ is complete among all valid blocks with $|T| \leq 4$, with no bound on $|L|$.*

The proof is short and is recorded in development notes that are not archived; it is not written out here, and the label rests on those notes. A brute-force test enumerated all pairs (T, L) with $|T| \leq 4$ and $L \subseteq \mathrm{Low}'$ for $m = 13$ (3,247,335 pairs, 25,106 valid), $m = 16$ (63,567,980; 70,412), $m = 17$ (164,886,060; 87,151), total about 2.3×10^8 pairs, and found no valid pair undominated by the family. The restriction $|T| \leq 4$ is a restriction of the searched family, not of the problem: a failure to find a cover in the family says nothing about covers with larger T.

Proposition 9 (PROVED; adversarial review recorded in development notes that are not archived)**.** *Drop every block (T, L) with $T = T_1 \sqcup T_2$, $L = L_1 \sqcup L_2$, all four nonempty, such that (T_1, L_1) and (T_2, L_2) are valid. The reduced family has a cover iff the full family does.*

The argument replaces a block by its two halves, each dominated by a kept block with smaller T, and terminates on $\sum |T|^2$. Reported effect (logged in development runs, not re-run for the archive; HEURISTIC): for $m = 49$, 1,992,263 blocks reduce to 26,427; for $m = 62$, 6.3×10^6 to 50,638. The reduction made the integer program tractable: before it, GLPK on the full family at $m = 47$ found no cover in about 12 minutes.

5 The verified values

Theorem 10 (VERIFIED per row)**.** *For every m in Table 1 or Table 2, and every set of m distinct integers in $[1, 2m]$,* ML $\geq 1/(m+1)$. *The Lean declaration is* `dense_floor_mm` *in the stated module. Together with Theorem 3, the dense window statement is Lean-verified for every m with $2 \leq m \leq 62$, and for every m with $63 \leq m \leq 126$ except $m \in E = \{94, 97, 103, 109, 115, 122\}$ (so $2m + 1 \in \{189, 195, 207, 219, 231, 245\}$). Hence the statement is* not *verified for all $m \leq 126$. For $m \in E$ no Lean module has been built. For $m \in \{97, 103, 109, 115, 122\}$ a certificate accepted by the independent checker* `v1` *is in* `data/dense/certs`*; this is COMPUTED and not VERIFIED. For $m = 94$ a search of three hours (*`code/dense/src/m94`*, log in* `data/dense/m94`*) ended without a cover; its exact phases stopped at their node caps, so the status is UNRESOLVED, and the run is a single logged run that was not repeated. It is not a non-existence result. No claim is made for these m. (The case $m = 1$ is elementary and not formalized here.) For $m \geq 64$ the declaration is* `dense_floor_mm_split`*.*

The statement for $m \leq 62$ is the union of Theorem 3 (all primes $2m + 1$) and the 33 composite values of $2m + 1$ in the tables (rows with $m \leq 62$). For $63 \leq m \leq 126$ there are 40 values with $2m + 1$ composite, 34 of which are rows of Table 2; the other six are E. The lists were compared by a script (`code/dense/check_rows.py`): the composite $m \in [2, 62]$ coincide with the union of the rows $m \leq 62$ and Table 1, and the composite $m \in [63, 126]$ not in the table are exactly E. The same script regenerated the parts, largest part and witness-time counts of the rows $m \geq 64$ from the Lean modules and compared parts and largest part with the output of the second checker.

Table 2: Cover-capacity certificates. Parts: number of parts U_i; max: largest part; times: number of witness times. In every row $\sum c_i = m - 1$. Modules carry the prefix `Delta2`. For $m \geq 64$ the modules are in split style (Remark 11); the row $m = 64$ is module `CoverRangeSB`, and $m = 62$ was additionally rechecked in split style as `CoverRangeSA`, same certificate. In these rows, times is the number N of witness times in the module, and the search entry "g1 local search, v1" means a Rust local search (`code/dense/src/g1`) followed by acceptance by the independent checker `code/dense/src/v1`. Build times and axiom prints for $m \geq 64$: `data/dense/results.tsv`, 45 to 176 s per module for $m \geq 66$, 56 s for $m = 64$. Search (provenance only: a certificate is accepted or rejected by the checkers independently of how it was found; the entries, including the times, are single logged runs of the development pipeline, HEURISTIC as statements about the searches, and were not re-run): ILP* is an integer program of the earlier pipeline whose tool and time are not recorded; pool+HiGHS is the early pool search (Python, HiGHS), n/r: time not recorded; CG is Rust column generation with a HiGHS master LP through Python; glpsol is the GLPK binary on a Rust-written model over the irreducible block family. Driver output of the CG and glpsol runs for $m \geq 27$ is in `data/dense/dev_logs`; no log was located for the ILP* rows. Axioms: S, a print equal to `propext, Classical.choice, Quot.sound` with zero `sorry` and no `native_decide` is archived as raw output (`data/dense/dev_logs` for $m \leq 64$, `data/dense/results.tsv` for $m \geq 66$); L, the same statement is recorded in `LEAN_MAP.md` and in development notes, and no raw output was located; –, not built. The raw prints in `dev_logs` cover `dense_floor_of_prime` and $m = 4, 7, 10, 12, 13, 16, 17, 19, 22$ (one Lean process on 2026-10-01 after building `Delta2Pigeonhole`, `Delta2DigitGreedy` and `Delta2DenseWindow`), and $m = 49, 52, 57, 58, 59, 60, 61, 62, 64$. No Lean build was run when the archive was assembled.

m	$2m+1$	parts	max	times	search	module	axioms
10	$3 \cdot 7$	5	6	14	ILP*	`CoverCapacity`	S
16	$3 \cdot 11$	7	6	22	ILP*	`CoverCapacity`	S
17	$5 \cdot 7$	6	9	26	ILP*	`CoverCapacity`	S
19	$3 \cdot 13$	6	9	27	ILP*	`CoverCapacity`	S
22	$3^2 \cdot 5$	7	8	31	ILP*	`CoverCapacity`	S
24	7^2	10	6	33	ILP*	`CoverCapacity`	L
25	$3 \cdot 17$	12	8	43	pool+HiGHS, ∼7 min	`CoverRangeI`	L

m	$2m+1$	parts	max	times	search	module	axioms
27	$5 \cdot 11$	17	8	41	pool+HiGHS, n/r	`CoverRangeJ`	L
28	$3 \cdot 19$	13	8	48	pool+HiGHS, ∼17 min	`CoverRangeK`	L
31	$3^2 \cdot 7$	17	12	51	CG, 1 s	`CoverRangeM`	L
32	$5 \cdot 13$	20	8	45	pool+HiGHS, 635 s	`CoverRangeL`	L
34	$3 \cdot 23$	18	12	54	CG, 62 s	`CoverRangeN`	L
37	$3 \cdot 5^2$	16	13	62	CG, 358 s	`CoverRangeO`	L
38	$7 \cdot 11$	23	10	55	CG, 4 s	`CoverRangeP`	L
40	3^4	22	10	61	CG, 35 s	`CoverRangeQ`	L
42	$5 \cdot 17$	25	10	61	CG, 10 s	`CoverRangeR`	L
43	$3 \cdot 29$	21	12	73	CG, 118 s	`CoverRangeS`	L
45	$7 \cdot 13$	28	8	64	CG, 5 s	`CoverRangeT`	L
46	$3 \cdot 31$	22	10	74	CG, 172 s	`CoverRangeU`	L
47	$5 \cdot 19$	26	10	67	CG, 32 s	`CoverRangeV`	L
49	$3^2 \cdot 11$	26	9	77	glpsol, 24.6 s	`CoverRangeAD`	S
52	$3 \cdot 5 \cdot 7$	23	9	85	glpsol, 74.1 s	`CoverRangeAE`	S
55	$3 \cdot 37$	25	11	90	CG, 521 s	`CoverRangeW`	L
57	$5 \cdot 23$	38	10	79	CG, 4 s	`CoverRangeX`	S
58	$3^2 \cdot 13$	25	9	97	glpsol, 6.3 s	`CoverRangeAF`	S
59	$7 \cdot 17$	29	10	88	CG, 100 s	`CoverRangeZ`	S
60	11^2	38	10	84	CG, 8 s	`CoverRangeAA`	S
61	$3 \cdot 41$	26	11	93	CG, 875 s	`CoverRangeAB`	S
62	5^3	44	8	81	glpsol (weighted), 15 min	`CoverRangeAC`	S
64	$3 \cdot 43$	23	9	108	g1 local search, v1	`CoverRangeSB`	S
66	$7 \cdot 19$	25	9	112	g1 local search, v1	`CoverRangeS66`	S
67	$3^3 \cdot 5$	26	9	110	g1 local search, v1	`CoverRangeS67`	S
70	$3 \cdot 47$	25	9	118	g1 local search, v1	`CoverRangeS70`	S
71	$11 \cdot 13$	26	9	120	g1 local search, v1	`CoverRangeS71`	S
72	$5 \cdot 29$	26	9	122	g1 local search, v1	`CoverRangeS72`	S
73	$3 \cdot 7^2$	29	8	120	g1 local search, v1	`CoverRangeS73`	S
76	$3^2 \cdot 17$	32	8	121	g1 local search, v1	`CoverRangeS76`	S
77	$5 \cdot 31$	27	9	130	g1 local search, v1	`CoverRangeS77`	S
79	$3 \cdot 53$	36	9	123	g1 local search, v1	`CoverRangeS79`	S
80	$7 \cdot 23$	28	9	135	g1 local search, v1	`CoverRangeS80`	S
82	$3 \cdot 5 \cdot 11$	38	8	129	g1 local search, v1	`CoverRangeS82`	S
84	13^2	27	9	146	g1 local search, v1	`CoverRangeS84`	S
85	$3^2 \cdot 19$	35	9	138	g1 local search, v1	`CoverRangeS85`	S
87	$5^2 \cdot 7$	31	9	147	g1 local search, v1	`CoverRangeS87`	S
88	$3 \cdot 59$	32	9	147	g1 local search, v1	`CoverRangeS88`	S
91	$3 \cdot 61$	36	8	149	g1 local search, v1	`CoverRangeS91`	S
92	$5 \cdot 37$	37	9	151	g1 local search, v1	`CoverRangeS92`	S
93	$11 \cdot 17$	31	9	160	g1 local search, v1	`CoverRangeS93`	S
100	$3 \cdot 67$	39	9	171	g1 local search, v1	`CoverRangeS100`	S
101	$7 \cdot 29$	37	9	169	g1 local search, v1	`CoverRangeS101`	S
102	$5 \cdot 41$	38	9	173	g1 local search, v1	`CoverRangeS102`	S
104	$11 \cdot 19$	39	9	174	g1 local search, v1	`CoverRangeS104`	S

m	$2m+1$	parts	max	times	search	module	axioms
106	$3 \cdot 71$	38	8	179	g1 local search, v1	`CoverRangeS106`	S
107	$5 \cdot 43$	37	9	183	g1 local search, v1	`CoverRangeS107`	S
108	$7 \cdot 31$	36	9	183	g1 local search, v1	`CoverRangeS108`	S
110	$13 \cdot 17$	38	9	187	g1 local search, v1	`CoverRangeS110`	S
112	$3^2 \cdot 5^2$	42	9	186	g1 local search, v1	`CoverRangeS112`	S
117	$5 \cdot 47$	43	9	196	g1 local search, v1	`CoverRangeS117`	S
118	$3 \cdot 79$	43	9	199	g1 local search, v1	`CoverRangeS118`	S
121	3^5	47	9	203	g1 local search, v1	`CoverRangeS121`	S
123	$13 \cdot 19$	40	9	214	g1 local search, v1	`CoverRangeS123`	S
124	$3 \cdot 83$	48	9	204	g1 local search, v1	`CoverRangeS124`	S
126	$11 \cdot 23$	41	9	216	g1 local search, v1	`CoverRangeS126`	S

Notes on the search record. The timings, node counts, seeds and correlations in these notes are single logged runs of the development pipeline; none was re-run for the archive, and as statements about the searches they are HEURISTIC. The certificates themselves do not depend on them. (i) The earlier pool-based pipeline used 64-bit masks. For $m \geq 33$ the shifts $1 \ll (x-1)$ wrap modulo 64, so every "infeasible" reported by that pipeline for $m \in \{34, 37, 38, 40, 42, 43, 45, 46, 55, 57, 58, 59\}$ is void. The certificates of rows $m \leq 32$ are unaffected, since each was re-checked by exact capacity and then proved in Lean. (ii) The certificates for $m = 49, 52, 58$ came from `glpsol`, a binary, run on a model written by Rust; the Rust depth-first search over the same family did not find them within the time spent ($m = 49$: about 6.5×10^7 nodes in about 1400 s) and is UNDECIDED there. (iii) The column-generation search used HiGHS from Python for the master LP and the integer step. (iv) Earlier failures at $m = 49, 52, 58$ were limits of the column pool, not properties of the method: all three have covers with $|T| \leq 4$. (v) A pure Rust local search (weighted, keeping the blocks disjoint on Y, on the irreducible $|T| \leq 4$ family) found covers for $m = 46, 49, 52, 58, 62$ without any external solver, in 4 to 44 seconds on two seeds (one $m = 52$ run among three extra seeds found nothing within 300 s, so the search is heavy-tailed there). Each cover passed the exact capacity check. These covers are COMPUTED and have not been built in Lean; the Lean modules in Table 2 use the earlier certificates. A local search cannot decide non-existence, and no non-existence claim is made. (vi) Time to the first cover correlates with the size of the block family (Spearman 0.84 for GLPK over 22 values of m) and only weakly with the arithmetic invariants tested; no conjecture is stated.

Remark 11 (Chunked checks, formalization device). In the split-style modules ($m = 62$ recheck and $m \geq 64$), the finite checks demanded by Theorem 5 are stated as separate lemmas, each closed by `decide +kernel`: one lemma each for the denominators, the cover and the capacity sum, windows of four witness times for the bad-set inclusions, and one lemma per part for the capacity condition, assembled by case distinction on the index. This is a device for keeping elaboration memory bounded. It changes neither the statements nor the certificates (the $m = 62$ certificate is the same in the monolithic module and in its split recheck) and carries no mathematical content. The data are plain lists; no `native_decide` and no `sorry` occur.

6 Reproducibility

All paths are relative to the root of the accompanying archive; `README.md` there gives the toolchain. The Rust tools are built by `bash code/dense/build.sh`. For $m \geq 64$ the pipeline is: `g1` (Rust local search, source `code/dense/src/g1`) writes `data/dense/certs/mM.g1.sol`; `v1 check M mM.g1.sol` (Rust, source `code/dense/src/v1`, no shared code with the search) must print ACCEPT; `lg2 gen2 M mM.g1.sol S<M> out` (source `code/dense/src/lg`) emits the Lean module; the modules are built one at a time and the axioms recorded in `data/dense/results.tsv`. The split style was introduced because the Lean build of the first module for $m = 64$, written in the monolithic style of the earlier rows, was killed by the operating system for memory after 282 s (exit 137); the $m = 62$ certificate built in 43 s in split style (monolithic: 130 s), and $m = 64$ in 56 s.

After the build, two scripts regenerate checkable content of Table 2 from the archive alone.

```
bash code/dense/build.sh build/dense
python3 code/dense/check_rows.py build/dense/v1
bash code/dense/regen_lean.sh build/dense
```

The second command reads the certificate (parts and capacities) out of each shipped Lean module, recomputes the columns parts, max and times, has `v1` accept the certificate independently, and compares the composite lists. The third regenerates the Lean modules for $m \geq 64$ from the certificates and compares them with the shipped modules. Both check exactly the data that the Lean kernel accepted. The output of a run is `data/dense/check_rows.out`. Two further programs regenerate content of the text. `code/dense/src/fb` regenerates Table 1 from the Lean source, and `code/dense/src/yind` regenerates the first sentence of Proposition 6 for $2 \leq m \leq 62$:

```
build/dense/fb check lean/LRSpectrumFull/Delta2FracBlocking.lean
build/dense/fb selftest lean/LRSpectrumFull/Delta2FracBlocking.lean
build/dense/yind 2 62
build/dense/yind control 13
```

What is not regenerated by a script in the archive: the search-method and time column for the rows $m \leq 62$ and the build times, which are logged values of single runs (partly archived as raw logs in `data/dense/dev_logs`); the raw axiom prints of the rows marked L; and the three-hour search for $m = 94$ (log archived, not repeated). The searches of the earlier pipeline are in `code/dense/src` (`pool`, `cg`, `xc`); their commands are

```
bin/cg run M 1 10 900 60 6000     # seed smax total_s mip_s attempts
bin/cg check M certs/mM.sol out.lean
bin/xc blocks M 4 6
bin/xc reduce M 4
IRR=1 code/dense/run_glp.sh M 4 2400 -99
bin/xc selftest M 4               # M <= 17
```

run from `data/dense`. Lean: one module at a time, `cd lean && lake build LRSpectrumFull.Delta2CoverRange<LETTER>`; axioms by `#print axioms LRSpectrumFull.dense_floor_m<M>` in a file with the matching `import`. A bare whole-tree `lake build` exhausts memory on a machine with 24 GB. Toolchain: Lean 4.30.0, Lake 5.0.0, Mathlib as pinned in `lean/lake-manifest.json`.

7 What is not claimed

- No new number of runners for the lonely runner conjecture. The window has height $2m$; the known reductions of the conjecture to finite checking require speeds of exponential size, so a statement at height $2m$ is a necessary special case at most.
- Nothing for m outside Theorem 3 and Tables 1, 2. No statement is made for $m \geq 127$ unless $2m+1$ is prime (Theorem 3). For $m \leq 126$ the values $m \in \{94, 97, 103, 109, 115, 122\}$ are not verified: no Lean module has been built (for $m = 94$ the search ended UNRESOLVED at its node caps, which is evidence about the search only; for the other five a certificate accepted by the independent checker exists and has not been built in Lean), so the dense window statement for all $m \leq 126$ is not claimed.
- Theorem 3 is a routine corollary of Tao's Lemma 5.1(i), the Bohman–Peng witness at modulus $2m+1$, and a two-line forcing step. The set of m with $2m+1$ prime has density zero, so it does not answer the question of [10] or Problem 6 of [5] for all m.
- A failed search is not evidence that no certificate exists (Section 5, notes (ii) and (iv)).
- For $m \leq 14$ the statement is subsumed by the lonely runner conjecture for up to 14 speeds announced in preprints (Section 8); for $m \geq 16$ with $2m+1$ composite I found no earlier statement, which is a search result and not a proof of novelty.

8 Prior art

Statements below were obtained by reading arXiv texts and abstract pages and by web searches; no Google Scholar or MathSciNet access was available.

Window and prime case. Tao [10] proves the floor for speeds in $[1, 1.2m]$ and remarks that $[1, 2m]$ is a natural target; Lemma 5.1(i) gives the divisibility statement used in Lemma 1. Bohman and Peng [3] prove, for m large, the strict inequality $1/(m+1) < \{v_i t\} < m/(m+1)$ for some t when $m < v_m \leq 2m - \exp(C(\log\log m)^2)$. In their proof, if s, t are coprime non-speeds with $2m - 2k < s + t \leq 2m$ and $q = s^{-1} \bmod (s+t)$, then the time $q/(s+t)$ is a witness. The argument uses only that s, t are not speeds, that $\gcd(s, s+t) = 1$, and that $s+t$ exceeds the largest speed; it therefore applies verbatim at $s + t = 2m + 1$, where for $2m+1$ prime coprimality is automatic. Bohman and Peng do not state this case (their hypothesis is $s + t \leq 2m$). Pomerance [6] states in the introduction that the interval $[1, 2m]$ remains a challenge; Pandey [4] treats speeds in arithmetic progressions and excludes exactly the progression with first term and difference both equal to 1. I read the arXiv version 1 of [3]; the published Mathematika text could not be read (HTTP 403 on the publisher site), Crossref lists no erratum, and a later remark in the published text could change the verdict on novelty. Pomerance's "Coprime matchings" was not read.

Small m. The conjecture for up to 14 speeds is claimed in [7, 11, 8, 9, 1]; [1] (version 2) is an unrefereed preprint whose computation I have not reproduced. This subsumes $m \leq 14$.

Method. The integral covering formulation of witness times (a tuple has no witness iff every allowed time is ruled out by some speed) is used computationally in [7, 11, 8, 9, 1]. I found no use of LP duality or capacity cuts for the lonely runner problem; the duality itself is textbook, so

the only possible novelty is the application. Status: HEURISTIC (absence of evidence from the searches recorded in the audit).

References

[1] Allikvere. Fourteen lonely runners, 2026. arXiv:2609.02604 (v2, 24 September 2026); not refereed as far as known.

[2] B. Bedert. Riesz products and the lonely runner conjecture: a wider gap of loneliness, 2025. arXiv:2511.16636.

[3] Tom Bohman and Fei Peng. Coprime mappings and lonely runners. *Mathematika*, 68:784–804, 2022. DOI 10.1112/mtk.12131; arXiv:2109.09860 (v1 only).

[4] Pandey. Title not recorded in the project sources. *Math. Bohem.*, 135:63–68, 2010.

[5] G. Perarnau and O. Serra. The lonely runner conjecture turns 60. *Comput. Sci. Rev.*, 58, 2025. Article 100798; arXiv:2409.20160.

[6] C. Pomerance. Title not recorded in the project sources. *Integers*, 22, 2022. Article A2.

[7] Matthieu Rosenfeld. The lonely runner conjecture holds for eight runners, 2025. arXiv:2509.14111.

[8] Matthieu Rosenfeld. Title not recorded in the project sources, 2025. arXiv:2512.01912.

[9] Sungkawichai and Trakulthongchai. Eleven, twelve, and thirteen lonely runners, 2026. arXiv:2604.23906.

[10] Terence Tao. Some remarks on the lonely runner conjecture. arXiv:1701.02048.

[11] Trakulthongchai. Nine and ten lonely runners, 2025. arXiv:2511.22427.